

Discussion 2: Transformation Analysis

Discussion Topic:

Transformation Analysis

Many computer vision applications use geometric transformations to change the position, orientation, and size of objects present in a scene. One such application where geometric transformations should be applied is counterfeit banknote detection.

Take a look at this image: Banknotes:



Import this image into OpenCV and examine its pixel matrix.

Discuss the translations you would perform to identify whether the banknotes are counterfeit and why you would apply them for counterfeit detection.

Describe how you would manually apply the transformations to this image's pixel matrix?

Discuss in detail your transformation matrices.

Apply your transformations and attach your transformed image to your post.

My Post:

Hello Class,

I imported the image into OpenCV using a Python script and used it to examine the image's pixel matrix and apply geometric transformations to the provided banknotes image. To compare the banknotes in the image with genuine bills, their position, orientation, and size need to be adjusted to align with the genuine bills' position, orientation, and size. Otherwise, a banknote feature on the provided image could be mistaken for a counterfeit feature simply because it is misaligned with the genuine reference bill. This comparison preprocessing is essential to obtain an accurate comparison between the provided banknotes and genuine bills. I am not rich enough to possess a \$100 bill, and no genuine images of a \$100 bill were provided; therefore, I only preprocessed the provided image and did not compare it with images of a genuine bill to determine if the bills in the image were counterfeit.

The image contains several bills, 3 total; they are hanging from a line with clips. I decided to clip from the image the regions containing the two bills in the foreground: the left region containing the back of a \$100 bill, and the right one containing the front of a \$100 bill; see Figure 1. Note that cropping, as well as image enhancement, are not part of geometric transformations; they are out of scope of this discussion. However, I decided to clip the image to isolate each foreground bill so that the geometric transformations could be applied to each bill separately.

Figure 1

Cropped Regions

Left foreground region



Right foreground region



Note. The images show the two regions cropped from the original image that were used for the geometric transformations.

To examine the image, I first loaded it using `cv2.imread()`, then I checked its shape using the following code lines:

```
image = cv2.imread(str(path), cv2.IMREAD_COLOR)
print(image.shape)    # (82, 124, 3)
print(image.dtype)    # uint8
```

The output was (82, 124, 3) and uint8.

This corresponds to 82 rows, 124 columns, and 3 BLUE, GREEN, and RED (BGR) color intensity channels. That gives $82 \times 124 = 10168$ pixels and $10168 \times 3 = 30,504$ color intensity channel values. OpenCV indexes an image by row first, so `image[20, 80]` accesses the point at $x = 80$ and $y = 20$ (OpenCV, n.d.-a). uint8 is an 8-bit integer with a value from 0 to 255 which also corresponds to RGB (red, blue, green) values; if an entered channel value is < 0 or > 255 OpenCV uses saturation arithmetic, meaning values below 0 are clipped to 0 and values above 255 are clipped to 255 (OpenCV, n.d.-b)

I use the following code clip/select the the two regions:

```
left_roi = image[0:82, 0:62].copy()
right_roi = image[0:82, 62:118].copy()
print(left_roi.shape)    # (82, 62, 3)
print(right_roi.shape)   # (82, 56, 3)
print(image[20, 80])     # BGR: [89, 109, 148]
```

The image BGR values and the pixel location vector are written in similar formats, but they represent two different things:

$$C = [B, G, R]$$

represents the color intensity stored at a pixel, while

$$P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

represents the pixel's position in the image. The transformation matrices operate on the position vector because geometric transformations change where a pixel is located, not directly what color that pixel contains. The extra 1 is a homogeneous coordinate. It makes it possible to include translation in the same matrix multiplication used for rotation and scaling. It is not a color channel or depth value (Colorado State University Global, n.d.).

For this image, I applied rotation, scaling, and translation. These are geometric transformations because they change the location of pixels in the image. The course material identifies scaling, rotation, and translation as basic image transformations (Colorado State University Global, n.d.).

Figure 2

Final Result Images

Left region: 90 degrees clockwise, 4x scale



Right region: 100 degrees clockwise, 4x scale



Note. Each output image is sized directly from the transformed coordinate bounds rather than being placed into a larger pre-sized image.

I rotated the left region 90 degrees clockwise and the right region 100 degrees clockwise because the two bills have different orientations in the original image. I then scaled both regions by a factor of 4; see Figure 2. The purpose of scaling here is to make the small 124 x 82 source image easier to examine. Enlarging the image does not create details that were not captured in the original photograph.

After rotation and scaling, I calculated the minimum and maximum transformed coordinates and sized each output directly from those bounds. Translation was then used only to shift transformed coordinates that would otherwise be negative into the valid output-image coordinate range. In an actual counterfeit-detection system, a further translation could be estimated during image registration to align

corresponding features with a genuine reference bill. Because no genuine reference image was supplied here, that reference-alignment step was not performed.

For each cropped region, I represent a pixel location using the homogeneous coordinate vector:

$$P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Because each crop has its own coordinate system, I build the transformation directly from the cropped region. The matrix C moves the crop center to the origin, R rotates it, S scales it by a factor of 4, and C^{-1} restores the crop center:

$$C = \begin{bmatrix} 1 & 0 & -c_x \\ 0 & 1 & -c_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$R = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$S = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$C^{-1} = \begin{bmatrix} 1 & 0 & c_x \\ 0 & 1 & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

After rotation and scaling, I calculate the minimum transformed x and y coordinates. The final translation shifts those minimum coordinates to (0,0), so the output image is only large enough to contain the transformed bill:

$$T_{fit} = \begin{bmatrix} 1 & 0 & -x_{min} \\ 0 & 1 & -y_{min} \\ 0 & 0 & 1 \end{bmatrix}$$

The complete transformation is:

$$M = T_{fit}C^{-1}SRC$$

In the Python script, the main calculations:

```
M = T_fit @ C_inv @ S @ R @ C # '@' matrix multiplication
```

For the left region, I used a 90-degree clockwise rotation and 4x scaling. For the right region, I used a 100-degree clockwise rotation and 4x scaling. The transformation is applied to each pixel position using:

$$P' = MP$$

When a transformed position falls between pixels, OpenCV uses interpolation to calculate the BGR value at the new location (OpenCV, n.d.-c). The transformed results are shown in Figure 2. These transformations prepare the bills for alignment with a genuine reference image, but because no genuine reference was provided, I did not perform counterfeit classification.

-Alex

References:

Colorado State University Global (n.d.). *Module 2: Image Formation* [Interactive Lecture]. Canvas. Page

OpenCV. (n.d.-a). *Basic operations on images*.
https://docs.opencv.org/4.13.0/d3/df2/tutorial_py_basic_ops.html

OpenCV. (n.d.-b.).). *Introduction*. OpenCV documentation.
<https://docs.opencv.org/4.13.0/d1/dfb/intro.html>

OpenCV. (n.d.-c). *Geometric image transformations*.
https://docs.opencv.org/4.13.0/da/d54/group__imgproc__transform.html